SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	T. Deekshith	**Reg. No.:**	192425145
Date:		**Duration:**	60 minutes

Code of Conduct

I ___T. Deekshith (192425145)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __T. Deekshith_____

Annexure A – Scenario-B ased Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**

- Carefully understand the given problem statement.
- Identify the business domain, stakeholders, existing challenges, and system objectives.

2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**

Develop an SRS document containing:

- Introduction
- Problem Description
- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

1. Analyze System Requirements

1.1 Detailed Problem Analysis

The Indian dairy industry plays an important role in the agricultural economy and provides income to millions of dairy farmers. However, diseases affecting dairy cattle create several challenges for milk production and farm management. Bovine mastitis is an important udder infection that affects the health and productivity of dairy animals.

Mastitis can occur in clinical and subclinical forms. Clinical mastitis may show visible symptoms such as swelling, redness, pain in the udder, and abnormal milk. Subclinical mastitis is more difficult to identify because visible symptoms may not be present, even though milk quality and production can be affected.

Traditional methods of detecting mastitis mainly depend on physical examination, milk testing, and veterinary consultation. These methods may not always identify the disease at an early stage, particularly in farms where regular veterinary services are difficult to access.

The proposed system uses Artificial Intelligence and Machine Learning to analyze animal health information and predict the possibility of mastitis. It considers different parameters, including milk production, body temperature, udder condition, milk quality, age, breed, and previous disease history.

The system processes these parameters using a trained predictive model and generates a risk category. Based on the prediction, farmers can identify animals that require additional monitoring and veterinary examination.

1.2 Scope of the Proposed System

The scope of the proposed system includes the development of an intelligent software platform for early mastitis risk forecasting.

The system focuses on the following activities:

1. Maintaining digital records of dairy animals.
2. Collecting milk production and animal health information.
3. Processing and analyzing collected data.
4. Identifying important mastitis-related risk factors.
5. Applying machine learning algorithms for risk prediction.
6. Classifying animals according to their predicted risk levels.
7. Generating alerts for animals requiring attention.
8. Maintaining historical health and prediction records.
9. Providing reports to farmers and veterinarians.
10. Supporting better decision-making in dairy farm management.

The system is designed as a decision-support tool. It does not replace laboratory testing, veterinary examination, or professional medical diagnosis.

1.3 Detailed System Objectives

1. Early Disease Identification

The main objective is to identify animals that may develop mastitis before the condition becomes severe. Early forecasting helps farmers take preventive steps and arrange veterinary examinations.

2. Intelligent Data Analysis

The system analyzes different animal-related parameters and identifies patterns that may indicate an increased risk of mastitis.

3. Improved Milk Production Management

Mastitis can reduce milk production and affect milk quality. Early identification helps farmers manage affected animals and reduce production losses.

4. Digital Record Management

The system maintains animal health records, milk production details, and historical prediction results in a centralized database.

5. Risk-Based Monitoring

The system classifies animals into different risk categories so that farmers can prioritize animals requiring additional observation.

6. Support for Veterinary Decisions

The prediction reports provide useful information to veterinarians when examining animals and deciding whether further tests are required.

7. Reduction of Economic Losses

Early identification and timely veterinary intervention can help reduce treatment costs, production losses, and other financial difficulties associated with mastitis.

8. Improved Dairy Farm Management

The system supports organized animal monitoring and encourages the use of digital technologies in traditional dairy farming.

1.4 Detailed Functional Requirements

Functional requirement	**Detailed description**
User registration	Farmers and veterinarians can create accounts and provide necessary details.

User login	Registered users can securely access the application.
Animal registration	Users can add animal ID, breed, age, and other details.
Data collection	The system collects milk yield, temperature, udder condition, and health information.
Data validation	The system checks whether the entered information is complete and valid.
Data preprocessing	Missing values and inconsistent data are handled before prediction.
Disease prediction	The trained ML model estimates the possibility of mastitis.
Risk classification	Animals are categorized into low, medium, and high-risk groups.
Alert generation	Notifications are generated for animals identified as high risk.
Report generation	The system creates reports containing animal details and prediction history.
Record management	Users can view and update animal information.
Historical analysis	Previous health records and prediction results can be reviewed.

Additional Functional Requirements

Data Input Management: The system should accept animal information through manual entry or supported connected devices. Input fields should be designed to collect relevant information in an organized manner.

Prediction History: Every prediction should be associated with the corresponding animal ID and date. This allows farmers to compare earlier and recent prediction results.

Veterinary Review: Veterinarians should be able to review animal information and prediction reports to support further examination.

Model Updating: The system should allow authorized developers to update the machine learning model when sufficient new and properly validated data becomes available.

1.5 Detailed Non-Functional Requirements

1. Scalability

The system should support the expansion of dairy farms, animal records, and user accounts. As the amount of collected data increases, the database and processing resources should be expandable without affecting the entire system.

2. Security

The system should maintain confidentiality of user information and animal health records. Password protection, authentication, authorization, and secure data transmission should be implemented.

3. Performance

The system should process input information and generate prediction results within an acceptable response time. Efficient algorithms and database queries should be used to reduce unnecessary processing delays.

4. Reliability

The system should provide consistent results when valid input data is supplied. Data validation, error handling, backup mechanisms, and regular model evaluation help improve system reliability.

5. Maintainability

The software should be designed in separate modules so that developers can modify individual components without affecting the complete application.

6. Availability

Farmers should be able to access the application whenever they need to check animal health information. Server monitoring and backup mechanisms can help maintain availability.

7. Usability

The interface should be simple, user-friendly, and suitable for farmers with different levels of technical knowledge. Clear labels, understandable alerts, and regional language support can improve usability.

8. Compatibility

The application should support commonly used web browsers and mobile devices. It should also allow integration with compatible sensors and external veterinary systems.

1.6 System Inputs and Outputs

The proposed system uses several input parameters to estimate mastitis risk.

Input parameters	Purpose
Animal identification	Identifies individual cattle.
Age	Helps analyze age-related health patterns.
Breed	Supports breed-related risk analysis.
Milk production	Helps identify changes in milk yield.
Body temperature	Provides information about animal health.
Udder condition	Helps identify possible inflammation-related symptoms.
Milk quality	Provides information about possible milk abnormalities.
Previous health history	Helps identify earlier disease patterns.

Expected system outputs:

1. Predicted mastitis risk category.
2. Animal health monitoring report.
3. High-risk animal notification.
4. Historical prediction details.

5. Animal-wise health summary.
6. Recommendations for veterinary examination.

The prediction accuracy depends on the quality, quantity, and representativeness of the training data.

2. Select a Suitable Software Architecture

2.1 Detailed Explanation of Selected Architecture

The proposed system follows a Three-Tier Layered Architecture integrated with an AI/ML prediction component. This architecture divides the complete application into separate functional layers, allowing each layer to perform specific operations.

The architecture is suitable for an AI-based dairy health monitoring application because it involves user interaction, data processing, machine learning prediction, database operations, and notification management.

The three main layers are:

1. Presentation Layer
2. Application Layer
3. Data Layer

The AI/ML prediction engine is integrated with the application layer as a specialized processing component.

2.2 Presentation Layer – Detailed Explanation

The presentation layer is the topmost layer of the architecture. It acts as the communication interface between the users and the software system.

Farmers and veterinarians use this layer to interact with the application. The interface allows users to enter animal information, view prediction results, check risk categories, and access historical records.

The presentation layer contains different screens and modules, such as:

- Registration and login page.
- Farmer dashboard.
- Animal registration page.
- Animal health data entry form.
- Milk production monitoring page.
- Prediction result page.
- Notification dashboard.
- Health report page.

The interface should be designed to provide clear information without requiring advanced technical knowledge. It can also support mobile devices to make the system accessible to farmers in rural areas.

2.3 Application Layer – Detailed Explanation

The application layer is the central processing layer of the system. It receives information from the presentation layer and performs the operations required to generate mastitis risk predictions.

This layer manages the main business logic and controls communication between the user interface, AI model, and database.

Its responsibilities include:

1. Validating user-entered information.
2. Managing user authentication and permissions.
3. Processing animal health information.
4. Sending prepared data to the machine learning model.
5. Receiving prediction results.
6. Classifying mastitis risk levels.
7. Generating alerts and reports.
8. Managing communication with the database.

The application layer ensures that information moves correctly between the different system components.

2.4 Data Layer – Detailed Explanation

The data layer is responsible for storing and managing all important system information. It maintains a centralized storage facility so that authorized users can access previously recorded information.

The database stores:

- Farmer registration information.
- Veterinarian information.
- Animal identification details.
- Breed and age information.
- Milk production records.
- Animal health measurements.
- Previous disease history.
- Machine learning prediction results.
- Notification records.

A relational database such as MySQL or PostgreSQL can be used to organize this information into related tables.

The data layer also supports data retrieval, updating, deletion according to permissions, and backup operations.

2.5 AI/ML Prediction Component – Detailed Explanation

The AI/ML prediction component is the intelligent part of the proposed system. It analyzes collected animal information and estimates the possibility of mastitis.

The model is trained using historical animal health information containing relevant input features and known disease outcomes.

Possible machine learning algorithms include:

- Logistic Regression.
- Decision Tree.
- Random Forest.
- Support Vector Machine.
- Gradient Boosting.

The trained model identifies patterns between animal health parameters and mastitis occurrence. When new animal information is entered, the model processes the data and produces a prediction score or risk category.

For example, changes in milk yield combined with other health-related indicators may suggest an increased risk. The system can classify the animal into an appropriate risk category for further observation.

The model should be evaluated using suitable performance measures such as accuracy, precision, recall, F1-score, and ROC-AUC. Recall is particularly important because missing animals that genuinely require further examination can have serious consequences.

2.6 Comparison of Different Software Architectures

Different architectural styles can be considered for developing the proposed system.

Architecture	Advantages	Limitations
Layered Architecture	Easy maintenance and clear separation of components.	Communication between layers may add overhead.
Client-Server Architecture	Supports centralized data processing and remote access.	Depends on server availability.
Microservices Architecture	Supports independent services and flexible scaling.	Requires complex deployment and management.
MVC Architecture	Separates data, user interface, and application control.	Does not independently define all backend infrastructure.
Event-Driven Architecture	Useful for real-time alerts and sensor events.	Event management can increase complexity.

2.7 Why Three-Tier Layered Architecture Is the Best Choice

The Three-Tier Layered Architecture is selected because the proposed system is an academic-level AI application that requires a clear separation of user interface, application processing, and data storage.

Unlike a complex microservices architecture, the layered approach is easier to implement, test, and maintain. It also allows the machine learning model to be integrated without requiring a completely separate infrastructure.

The architecture supports future improvements such as IoT sensor integration, mobile applications, cloud storage, and advanced prediction models.

Therefore, the selected architecture provides a balanced solution between development simplicity, performance, scalability, and maintainability.

3. Draw the Software Architecture Diagram

3.1 Detailed Architecture Design

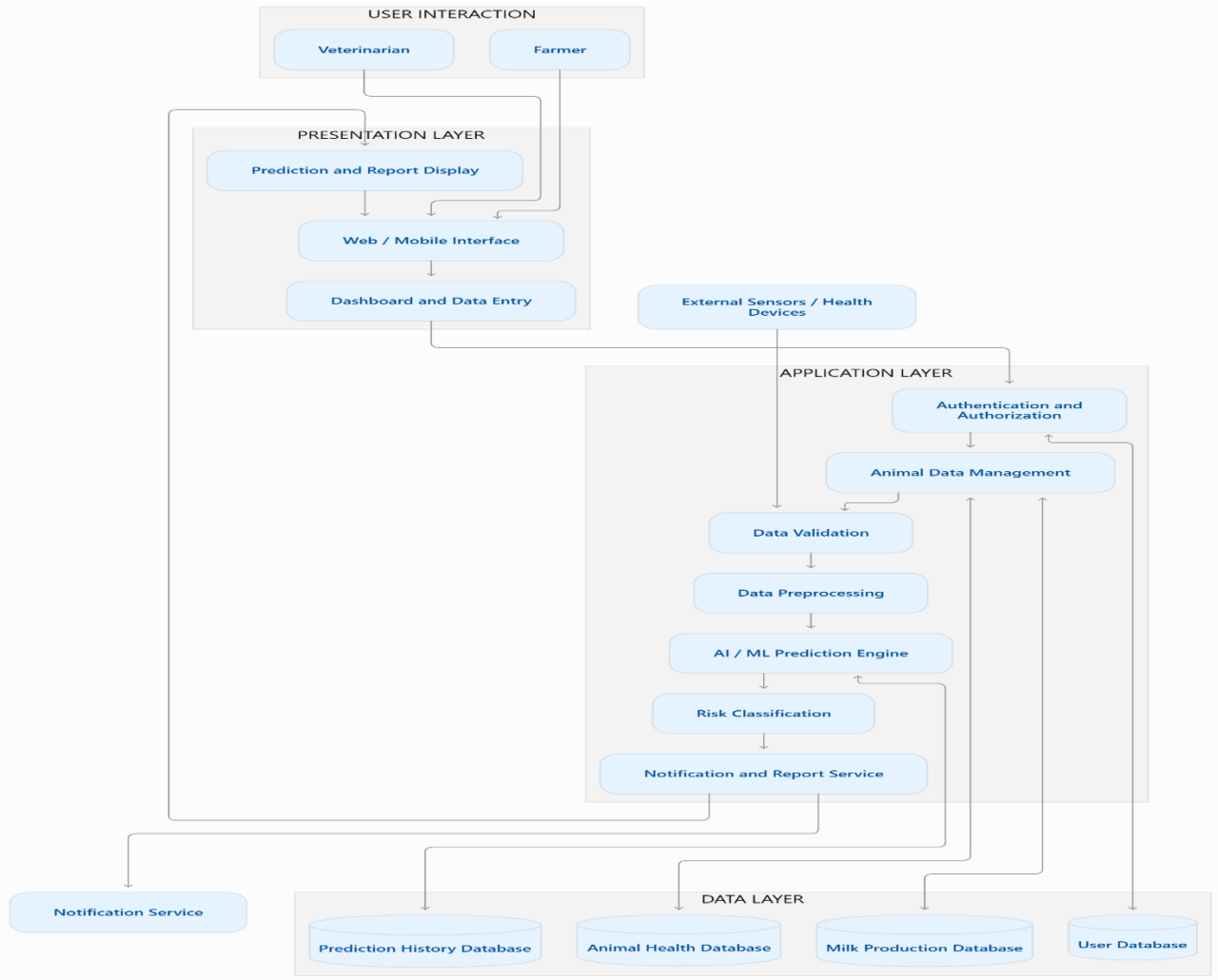
The architecture design represents the complete structure of the proposed AI-based mastitis prediction system. It explains how different components are organized and how information flows between them.

The system begins when a farmer or veterinarian accesses the application. The user enters animal details and health information through the presentation layer.

The application layer receives the information and performs validation and preprocessing. The prepared information is then transferred to the AI/ML prediction engine, which analyzes the data using a trained machine learning model.

After prediction, the system classifies the result into different risk levels. The prediction information is stored in the database and displayed on the user's dashboard. If an animal is identified as high risk, the system generates an alert recommending further veterinary examination.

3.2 Detailed Architecture Diagram



3.3 Explanation of Data Flow in the Architecture

Step 1 – User Access: The farmer or veterinarian opens the application using a web browser or mobile device.

Step 2 – Authentication: The system verifies the login credentials and provides access based on the user's role.

Step 3 – Data Collection: The farmer enters animal information, including milk production, body temperature, udder condition, and previous health records.

Step 4 – Data Validation: The application checks the entered values and identifies missing or invalid information.

Step 5 – Data Preprocessing: The information is cleaned, transformed, and prepared in the format required by the machine learning model.

Step 6 – AI Prediction: The trained model analyzes the prepared information and estimates the mastitis risk.

Step 7 – Risk Classification: The system converts the prediction into low, medium, or high-risk categories based on configured thresholds.

Step 8 – Database Storage: The prediction results and relevant animal information are stored in the database.

Step 9 – Notification Generation: If the risk is high, the system generates an alert for the farmer and recommends veterinary assessment.

Step 10 – Result Presentation: The application displays prediction results, historical records, and reports on the dashboard.

3.4 Database Design Overview

The database is an important component because the proposed system requires continuous storage and retrieval of animal health information.

The following tables can be used:

Table name	Important fields
Farmer	Farmer_ID, Name, Email, Phone
Animal	Animal_ID, Farmer_ID, Breed, Age
Health_Record	Record_ID, Animal_ID, Temperature, Udder_Condition
Milk_Record	Milk_ID, Animal_ID, Milk_Yield, Milk_Quality
Prediction	Prediction_ID, Animal_ID, Risk_Level, Prediction_Date
Notification	Notification_ID, Animal_ID, Message, Status

The Farmer_ID connects farmers with their registered animals. The Animal_ID connects individual animals with their health records, milk production records, and prediction history.

This relational structure reduces unnecessary duplication and makes it easier to retrieve animal-related information.

4. Explain Components and Their Interactions

4.1 Detailed Component Responsibilities

1. User Management Component

The user management component handles the registration, login, and account management operations of the system. It allows different categories of users to access the application according to their responsibilities.

Farmers can manage animal records and monitor prediction results, while veterinarians can review health information and assist with further examination.

2. Animal Management Component

This component maintains information about individual dairy animals. Each animal is assigned a unique identification number to distinguish it from other animals in the farm.

The component stores details such as animal breed, age, lactation information, identification number, and farm details.

3. Health Data Collection Component

The health data collection component gathers information that may be useful for identifying mastitis risk.

It can collect data through manual entry or supported monitoring devices. Information such as body temperature, udder condition, milk appearance, and previous health problems can be recorded.

The collected data becomes the primary input for the prediction process.

4. Milk Production Monitoring Component

Milk production is an important parameter for understanding the productivity and health condition of dairy cattle.

This component records daily milk yield and other available milk quality information. Changes in milk production over time can be analyzed along with other animal health indicators.

5. Data Preprocessing Component

Raw data collected from dairy farms may contain missing values, duplicate entries, incorrect measurements, or inconsistent formats.

The preprocessing component performs the following operations:

- Removing duplicate records.
- Handling missing values.
- Checking incorrect data entries.
- Converting data into suitable formats.
- Normalizing numerical features when required.
- Preparing input features for the prediction model.

This process helps improve the consistency of the data supplied to the AI model.

6. Machine Learning Component

The machine learning component is responsible for identifying patterns in historical animal health information.

It uses a trained model to analyze input parameters and estimate the probability of mastitis. Different algorithms can be tested to determine which provides the most suitable performance on the available dataset.

The selected model is integrated into the application and used to generate predictions for newly entered animal information.

7. Risk Analysis Component

The risk analysis component interprets the prediction output and converts it into understandable categories.

For example:

- Low Risk: No strong indication of increased mastitis risk based on the available data.
- Medium Risk: Some indicators require closer monitoring and possible additional testing.
- High Risk: The animal requires prompt veterinary assessment.

The categories should be determined using validated model thresholds rather than arbitrary values.

8. Notification Component

The notification component communicates important prediction results to users.

When an animal is classified as high risk, the system generates a notification containing the animal identification number, risk category, prediction date, and recommended next action.

Notifications can be displayed within the application and may also be delivered through supported SMS or email services.

9. Reporting Component

The reporting component prepares structured reports containing animal health details and prediction history.

Reports can include daily milk production, previous health observations, risk classifications, and monitoring records.

These reports help farmers understand changes in animal health over time.

10. Database Component

The database component maintains the information required by all other system components.

It supports data insertion, retrieval, updating, and authorized deletion. It also maintains historical records that can be used for future analysis and machine learning model improvement.

4.2 Detailed Component Interaction

The components communicate through well-defined interfaces and exchange information in a systematic manner.

The user interface sends requests to the application layer. The application layer validates the information and communicates with the database to retrieve relevant animal records.

After retrieving the required information, the data processing component prepares the input features and transfers them to the machine learning prediction engine.

The prediction engine returns a prediction score or result. The risk analysis component interprets this result and determines the appropriate risk category.

The notification component receives the risk information and generates alerts whenever necessary. Finally, the reporting component presents the prediction information to farmers and veterinarians.

This interaction ensures that each component performs its assigned task without unnecessary dependency on other components.

4.3 Communication Mechanisms

The proposed system can use the following communication mechanisms:

Communication mechanism	Purpose
HTTP/HTTPS	Communication between the web or mobile interface and backend.
REST API	Exchange of animal information and prediction requests.
SQL Queries	Retrieving and storing information in the database.
JSON	Transferring structured data between application components.
Model API	Sending processed information to the ML model and receiving predictions.
SMS/Email API	Sending alerts to farmers and veterinarians.

These mechanisms provide structured communication and allow the system to integrate with external services in the future.

4.4 Example of System Operation

Consider a dairy farm containing 50 cows. The farmer regularly records milk production and health information for each animal.

One day, the farmer notices a reduction in milk production in one cow and enters the latest health observations into the system.

The system retrieves the animal's previous records and combines them with the newly entered information. The preprocessing component prepares the data and sends it to the trained machine learning model.

The model estimates the risk of mastitis and the application classifies the result as high risk.

An alert is displayed on the farmer's dashboard, recommending veterinary examination and appropriate diagnostic testing.

The farmer can then share the animal's history and prediction report with a veterinarian. This process supports earlier investigation and helps prevent delays in identifying possible disease conditions.

5. Discuss Quality Attributes

Quality attributes describe how effectively the proposed software system performs its operations. They help determine whether the system is secure, reliable, efficient, scalable, and suitable for practical use.

5.1 Scalability

Scalability refers to the ability of a software system to handle an increase in users, data, and processing requirements without affecting its overall performance.

The proposed mastitis prediction system is designed to support dairy farms of different sizes. Initially, the system may be used by a small number of farmers managing a limited number of animals. In the future, it can be expanded to support hundreds or thousands of farms.

As the number of registered animals increases, the database can be expanded to store additional health records, milk production details, and prediction results.

The machine learning component can also be improved to handle larger datasets and more complex prediction tasks.

Methods used to improve scalability:

1. Using cloud-based database services for storing large amounts of information.
2. Increasing server capacity when the number of users grows.
3. Optimizing database queries to retrieve information efficiently.
4. Designing independent software modules.
5. Supporting multiple farms through a common application.
6. Using efficient machine learning models for large datasets.

Thus, scalability allows the system to expand according to the requirements of the Indian dairy industry.

5.2 Security

Security is an essential quality attribute because the system stores farmer information, animal health records, and other important farm-related details.

The proposed system uses authentication and authorization mechanisms to ensure that only registered and permitted users can access the application.

Different user roles can be created to control access to specific functions. For example, farmers can manage their own animal information, while veterinarians can review relevant animal health records.

The system should also protect information during communication between the user interface, application server, and database.

Security measures include:

1. Secure user registration and login.
2. Password hashing for protecting login credentials.
3. Role-based access control.
4. HTTPS-based communication.
5. Input validation to prevent unauthorized or harmful inputs.
6. Regular database backups.
7. Secure database access.
8. Protection against unauthorized modification of records.

These measures help maintain the confidentiality, integrity, and security of stored information.

5.3 Performance

Performance refers to how quickly and efficiently the system processes information and responds to user requests.

The proposed system must process animal health information and generate mastitis risk predictions without unnecessary delays.

When a farmer enters animal details, the application validates the information and sends it to the prediction component. The machine learning model analyzes the data and returns the prediction result.

The system should also provide quick access to previous animal records and prediction reports.

Methods used to improve performance:

1. Using efficient machine learning algorithms.
2. Optimizing data preprocessing operations.
3. Reducing unnecessary database queries.
4. Using suitable database indexing.
5. Avoiding repeated processing of unchanged information.
6. Optimizing application programming interfaces.
7. Using sufficient server resources.

Good performance improves the user experience and makes the system more useful for daily farm monitoring.

5.4 Reliability

Reliability refers to the ability of the system to operate correctly and consistently under expected conditions.

The proposed system should provide dependable results whenever valid animal health information is supplied.

Prediction reliability depends on the quality of training data, the selected machine learning algorithm, and proper model evaluation.

The system should also handle errors such as invalid input, database connection failures, and unavailable prediction services.

Methods used to improve reliability:

1. Validating animal health information before processing.
2. Using properly trained machine learning models.
3. Testing the prediction model with separate validation and test datasets.
4. Performing regular system testing.
5. Maintaining backup copies of important records.
6. Implementing error-handling mechanisms.
7. Monitoring application performance.
8. Evaluating prediction results periodically.

These methods help reduce system failures and improve the consistency of its operations.

5.5 Maintainability

Maintainability refers to the ease with which software can be modified, updated, corrected, and improved after development.

The Three-Tier Layered Architecture supports maintainability by separating the presentation, application, and data layers.

For example, if developers want to improve the prediction algorithm, they can update the machine learning component without completely changing the user interface or database structure.

Similarly, additional features such as multilingual support, sensor integration, and advanced reporting can be added through suitable modifications.

Methods used to improve maintainability:

1. Dividing the application into separate modules.
2. Following consistent coding standards.
3. Maintaining proper technical documentation.
4. Performing regular software testing.
5. Using version control systems.
6. Updating machine learning models when required.
7. Separating configuration settings from application logic.
8. Monitoring and correcting software errors.

This approach reduces maintenance difficulties and supports the long-term development of the system.

5.6 Availability

Availability refers to the ability of the system to remain accessible whenever users require its services.

Farmers may need to access animal health information at different times of the day. Therefore, the proposed application should provide reliable access to the dashboard, animal records, and prediction services.

A cloud-based deployment can help provide remote access from different geographical locations.

The system should also include recovery mechanisms to reduce the impact of server failures and unexpected interruptions.

Methods used to improve availability:

1. Using reliable hosting services.
2. Maintaining regular database backups.
3. Monitoring server health.
4. Implementing recovery mechanisms.
5. Using proper error handling.
6. Providing access through mobile-friendly interfaces.
7. Maintaining alternative procedures for recording information during temporary network failures.

These measures help ensure that farmers can access important animal health information when required.

5.7 Summary of Quality Attributes

Quality attribute	Implementation in the proposed system
Scalability	Supports additional farms, users, and animal records.
Security	Uses authentication, authorization, and secure communication.
Performance	Uses optimized data processing and prediction algorithms.
Reliability	Uses validation, testing, backups, and model evaluation.
Maintainability	Uses modular components and layered architecture.
Availability	Uses reliable hosting, monitoring, and recovery mechanisms.

6. Justify Architectural Decisions

6.1 Detailed Rationale Behind Architectural Decisions

The selection of an appropriate software architecture is important for the successful development of the proposed AI-based mastitis prediction system.

The system requires different functionalities, including user management, animal data collection, data preprocessing, machine learning prediction, database management, and notification generation.

The Three-Tier Layered Architecture is selected because it provides a structured method for organizing these functionalities.

The presentation layer handles communication with farmers and veterinarians. The application layer manages business operations and coordinates the prediction process. The data layer stores and retrieves the information required by the application.

The machine learning prediction engine is integrated into the application layer to analyze the collected animal information.

This separation reduces unnecessary dependencies between components and makes the application easier to develop, test, and maintain.

6.2 Reasons for Selecting Layered Architecture

1. Separation of Responsibilities

Each layer performs a specific function. The presentation layer handles user interaction, the application layer performs processing, and the data layer manages information storage.

2. Easy Development

Developers can work on different layers independently. This helps organize the development process and reduces confusion during implementation.

3. Easy Maintenance

If one component requires modification, developers can update that component without changing the entire system, provided its interfaces remain compatible.

4. Better Data Management

The centralized database helps maintain organized animal health records and prediction history.

5. AI Model Integration

The architecture supports the integration of machine learning models for analyzing animal information and forecasting mastitis risk.

6. Future Expansion

New functionalities such as IoT sensors, mobile applications, and veterinary integration can be added without completely redesigning the application.

6.3 Trade-Offs Considered While Selecting the Architecture

Every software architecture has advantages and limitations. The proposed architecture was selected after considering the following trade-offs.

1. Simplicity vs. Flexibility

A simple architecture makes development easier, but it may provide limited flexibility when the system becomes very large.

The layered approach provides a balance by separating major responsibilities while keeping communication between components understandable.

2. Performance vs. Separation

Separating the system into multiple layers may introduce additional communication overhead.

However, this separation improves code organization, testing, and maintenance, which are important for long-term development.

3. Centralized Database vs. Distributed Storage

A centralized database simplifies information management and makes animal records easier to access.

However, as the system grows, database capacity and performance must be monitored carefully.

4. Manual Data Entry vs. Automatic Data Collection

Manual data entry is less expensive and easier to implement in small dairy farms.

However, it may introduce errors and require additional effort from farmers. Automatic data collection through IoT sensors can improve efficiency but increases equipment and implementation costs.

5. Prediction Accuracy vs. Processing Requirements

Complex machine learning algorithms may identify more complicated patterns, but they may require additional processing resources.

The selected model should balance prediction performance, computational requirements, and the quality of available training data.

6.4 Technical Technologies Used in the Proposed System

The following technologies can be considered for implementing the proposed architecture.

Component	Suggested technology	Purpose
Frontend	HTML, CSS, JavaScript	Creating the user interface.
Frontend framework	React.js	Building interactive dashboards.
Backend	Python Flask or Django	Managing application logic.
Machine Learning	Python, Scikit-learn	Developing prediction models.
Data processing	Pandas, NumPy	Cleaning and preparing data.
Database	MySQL	Storing animal and farmer records.
Visualization	Matplotlib, Chart.js	Displaying prediction and monitoring reports.
API communication	REST API	Connecting different components.
Deployment	Cloud hosting	Providing remote application access.

These are suggested implementation technologies. The architecture does not depend on using all of them.

6.5 Future Enhancements

The proposed system can be improved through the addition of advanced technologies and new functionalities.

1. IoT-Based Animal Health Monitoring

In the future, IoT sensors can be connected to dairy animals to collect health information automatically. Sensors can monitor body temperature, activity levels, and other measurable parameters.

The collected information can be transferred to the application through suitable communication technologies.

This reduces manual data entry and supports more frequent animal monitoring.

2. Advanced Machine Learning Algorithms

Future versions can implement advanced algorithms such as Random Forest, XGBoost, Support Vector Machine, and Neural Networks.

Different models can be compared using suitable evaluation measures to identify the most appropriate algorithm for the available dataset.

3. Mobile Application Development

A dedicated Android or iOS application can be developed to help farmers access animal records and prediction results using smartphones.

The application can provide alerts and allow farmers to enter health information directly from the farm.

4. Regional Language Support

Indian dairy farmers speak different regional languages. Therefore, the system can support languages such as Tamil, Telugu, Hindi, Kannada, and Malayalam.

Regional language support can make the application easier to understand and improve adoption among rural farmers.

5. Cloud-Based Data Management

Cloud computing can be integrated to store large amounts of animal health information and prediction records.

It allows authorized users to access the application from different locations and supports future expansion.

6. Real-Time Disease Risk Monitoring

Future versions can analyze continuously collected health information and identify unusual changes in animal conditions.

This can help farmers monitor risk changes more frequently instead of depending only on occasional manual data entry.

7. Veterinary Consultation Integration

The system can provide a platform for sharing animal health records and prediction reports with veterinarians.

Veterinarians can review the information, conduct further examinations, and provide appropriate guidance.

8. Regional Disease Pattern Analysis

The system can analyze aggregated and appropriately protected data from multiple dairy farms to identify geographical and seasonal patterns associated with mastitis risk.

Such analysis may help researchers and dairy farm administrators understand disease trends and plan preventive activities.

6.6 Long-Term Maintainability and System Integration

The proposed architecture supports long-term maintenance by keeping the major system functionalities separate.

The prediction component can be upgraded when new training data and improved machine learning algorithms become available. The presentation layer can be redesigned to support new devices and languages.

External systems can be connected through APIs, allowing integration with veterinary platforms, IoT monitoring devices, and cloud services.

The database structure can also be expanded to store additional animal information and historical records.

These capabilities make the architecture suitable for future improvements and long-term use in dairy farm management.

7. Conclusion

The AI-Based Predictive Modelling for Early Forecasting of Bovine Mastitis in Indian Dairy Farms is a proposed intelligent software solution that aims to improve the monitoring and management of dairy animal health.

Bovine mastitis affects milk production, milk quality, and the economic condition of dairy farmers. Traditional identification methods may face difficulties in detecting subclinical cases at an early stage. The proposed system addresses this challenge by applying Artificial Intelligence and Machine Learning techniques to analyze animal health information and estimate the possibility of mastitis.

The system collects information such as animal age, breed, milk production, body temperature, udder condition, and previous health records. It processes the collected information using a trained machine learning model and generates risk classifications. Farmers can use these results to identify animals requiring additional observation and veterinary examination.

The Three-Tier Layered Architecture provides an organized structure for implementing the proposed system. It separates user interaction, application processing, and data storage into different layers. The integration of AI/ML prediction, risk classification, notification generation, and database management provides a systematic approach to early disease forecasting.

The proposed architecture also addresses important software quality attributes such as scalability, security, performance, reliability, maintainability, and availability. Future improvements involving IoT sensors, cloud computing, mobile applications, and advanced machine learning algorithms can further enhance the system.

Overall, the proposed system can support farmers and veterinarians in making informed decisions, encourage early veterinary intervention, improve animal health monitoring, and contribute to better dairy farm management in India.

8. References

You can include the following references in the assignment:

1. Sommerville, I., Software Engineering, 10th Edition, Pearson Education.
2. Pressman, R. S., and Maxim, B. R., Software Engineering: A Practitioner's Approach, McGraw-Hill.
3. Bishop, C. M., Pattern Recognition and Machine Learning, Springer.
4. Hastie, T., Tibshirani, R., and Friedman, J., The Elements of Statistical Learning, Springer.
5. Scikit-learn Documentation – Machine Learning in Python.
6. Food and Agriculture Organization (FAO) – Dairy Production and Animal Health Resources.
7. Indian Council of Agricultural Research (ICAR) – Animal Health and Dairy Research Resources.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case	Correctly identifies all relevant actors and use cases; use	Most actors and use cases identified;	Limited actors/use cases identified;	Actors/use cases missing or incorrect;

Modelling (20%)	case diagram is complete, accurate, and follows UML standards.	diagram has minor errors.	diagram partially correct.	diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5

Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25